LogiEdge Phase 2 Report: Model Lifecycle and Edge MLOps

AIML ZG535 Machine Learning on Edge (EC-I)

Group 15

Student Details:

BITS ID	Name	Contributions
2025ae05381	Disha Shetty H	100%
2025ae05382	Murali Krishna Sai Garisapati	100%
2025ae05420	Rajan Yadav	100%
2025ae05680	Santosh Kumar Behera	100%

Repository: https://github.com/santosh-2025ae05680/logibridge

1. Scope

This report covers Components D and E (Modules 4 and 5) and the mandatory normalisation experiment of Task C2: dataset generation and model training, TFLite conversion, the experiment with statistics shifted by 3σ, Docker containerisation with the OTA layer-cache demonstration, PSI drift monitoring, Ansible deployment and OTA strategy selection. Model optimisation (pruning and benchmarking) is reported in the final report. All data is simulated.

2. Dataset generation and model training (Task D1)

The dataset is generated in virtual time from the simulator's own generator and the same preprocessing code the live service uses, so training and serving share one code path. Seeds are fixed (101, 102 and 103 for the three classes); the normalisation statistics used seed 42, so the two never share random draws. Windows overlap by 20 s, so a random 80/20 split would put near-duplicates of training windows into the validation set and inflate accuracy. Instead, the last 20% of each class in time order is the validation set, and two windows between training and validation are purged so that no raw samples are shared.

Table 1: Dataset

Class	Simulator mode	Duration	Windows	Train	Validation	Purged
0 Normal	none	20 min	117	92	23	2
1 Warning	temp_drift	15 min	87	68	17	2

Class	Simulator mode	Duration	Windows	Train	Validation	Purged
2 Critical	combined	15 min	87	68	17	2

This gives 291 windows generated, of which 228 are used for training and 57 for validation. The model is a multilayer perceptron with layers 6-32-16-3 (ReLU hidden layers, softmax output, 803 parameters), trained with Adam (learning rate 1e-3), batch size 16, 60 epochs and sparse categorical cross-entropy, on features normalised with `training_stats.npy`. Training accuracy reached 100% by epoch 12, and validation accuracy first reached 100% at epoch 9.

Table 2: Validation confusion matrix (rows true, columns predicted)

True class	Normal	Warning	Critical	Recall
Normal	23	0	0	1.000
Warning	0	17	0	1.000
Critical	0	0	17	1.000

Validation accuracy is 100% on 57 windows, against the 88% gate, and Critical recall is 100%. **This is not evidence of real-world accuracy.** After clipping, every anomalous window has the same normalised temperature features, and Warning and Critical differ only in the vibration features, so the simulated classes are separated by more than 10σ.

Table 3: Model confidence by true class (validation)

True class	Mean max-softmax	Mean P(Normal)
Normal	0.995	0.995
Warning	1.000	0.000
Critical	0.999	0.000

Table 3 matters for drift monitoring: the maximum softmax probability is about 1.0 for every class, so it cannot reveal a fault, whereas the probability of the Normal class can (Section 7).

3. TFLite conversion

Table 4: Converted models

Variant	File size	Validation accuracy	Critical recall
Keras FP32	not applicable	100.00%	100.00%
M1 TFLite FP32	5.19 KB	100.00%	100.00%
M2 TFLite INT8 (PTQ)	3.38 KB	100.00%	100.00%

M2 uses full-integer post-training quantisation with int8 input and output, calibrated on the 228 training windows only (the brief asks for at least 200); validation data is never used for calibration. M2 is 35% smaller than M1 and agrees with the FP32 model on all 285 windows; the largest probability difference is 0.0039, one output step (scale 1/256, zero point -128). The input quantiser (scale 0.0533, zero point -61) covers z-scores from about -3.57 to +10.02, which the ±10 clip bounds.

4. Mandatory experiment: statistics shifted by 3σ (Task C2)

Method. The same validation windows are classified with the correct statistics and with statistics shifted by +3σ and -3σ, which shifts every normalised feature by -3 or +3 respectively. Both the FP32 Keras model and the INT8 model are tested, with the ±10 clip on (the deployed configuration) and off.

Table 5: Accuracy under shifted statistics (57 validation windows)

Model	Clip	Statistics	Accuracy	Recall % (Normal / Warning / Critical)
FP32 Keras	±10	correct	100.0%	100.0 / 100.0 / 100.0
FP32 Keras	±10	+3σ	100.0%	100.0 / 100.0 / 100.0
FP32 Keras	±10	-3σ	100.0%	100.0 / 100.0 / 100.0
FP32 Keras	off	correct	40.4%	100.0 / 0.0 / 0.0
FP32 Keras	off	+3σ	40.4%	100.0 / 0.0 / 0.0
FP32 Keras	off	-3σ	40.4%	100.0 / 0.0 / 0.0
INT8 TFLite	±10	correct	100.0%	100.0 / 100.0 / 100.0
INT8 TFLite	±10	+3σ	100.0%	100.0 / 100.0 / 100.0
INT8 TFLite	±10	-3σ	100.0%	100.0 / 100.0 / 100.0
INT8 TFLite	off	correct	100.0%	100.0 / 100.0 / 100.0
INT8 TFLite	off	+3σ	100.0%	100.0 / 100.0 / 100.0
INT8 TFLite	off	-3σ	100.0%	100.0 / 100.0 / 100.0

In raw units, 3σ is 0.133 °C for temperature mean, 0.054 °C for temperature standard deviation, 0.79 °C/min for the rate of change, 0.038 g for vibration RMS, 0.056 g for vibration peak and 2.53 for kurtosis.

Findings. (1) In the deployed configuration a 3σ shift changes nothing in either direction. The shift moves Normal windows by 3 in z, but the nearest anomaly is more than 10 away at the clip; real, overlapping faults would be far more sensitive. (2) With the clip off, the FP32 model falls to 40.4%, which is exactly the 23 Normal windows out of 57, so every Warning and Critical window is misclassified regardless of the shift. Unclipped z-scores in the hundreds lie far outside the range the network trained on, where ReLU extrapolation is unpredictable. (3) The INT8 model stays at 100% without the clip because its input quantiser saturates at the calibrated range, which acts as an implicit clip.

Conclusion. The clip and the statistics are part of the model's input contract. They must be versioned with the model and shipped with it (baked into the image), and never recomputed from live data.

5. Docker containerisation (Task D2)

The inference service subscribes to the temperature and vibration topics, closes windows using message timestamps (restarting windowing after a timestamp jump back of more than 5 s or a gap above 2 minutes), normalises with the frozen statistics, classifies with a TFLite interpreter (`tflite-runtime` 2.14.0 in the container) and publishes each result to `logibridge/trucks/{truck_id}/inference`. The result carries the timestamp, class, label, confidence, P(Normal), all probabilities, the inference latency and a model tag made of the file name and a hash. An alert is raised on entering or escalating Warning or Critical and repeated every 60 s while it persists; it is published with QoS 1 and appended to a local JSON-lines log. Configuration is by environment variables (`MODEL_PATH`, `TRUCK_ID`, `MQTT_HOST`, `MQTT_PORT`). The broker runs as its own container on a user-defined network with its port published to 127.0.0.1 only.

Table 6: Dockerfile layers, in order

Layer	Content	Size (uncompressed)
Base	python:3.11-slim	about 125 MB
Dependencies	requirements.txt and pip install	239 MB
Code	preprocessing, TFLite wrapper, inference service	about 20 KB
Statistics	training_stats.npy	224 bytes
Metadata	ENV and CMD	0 bytes
Model	COPY model.tflite (last)	3.46 KB

All pip layers come before the model copy, and the metadata layers sit before it so that the model is the very last instruction: an update then rebuilds and ships exactly one layer. The whole image is about 364 MB uncompressed (120.65 MB compressed in the registry), and the running service used 61.5 MiB of the 7.6 GiB available while idle. In the final M3 deployment, the first inference-service log reports model.tflite@84ff2345 with input=int8.

6. OTA layer-cache demonstration

Only the model file was replaced and the image rebuilt. Every build step printed CACHED except the final copy of the model, and pushing the new tag to the local registry sent one layer while the other ten reported that the layer already exists.

Table 7: Bandwidth for 85 trucks

Update method	Per truck	85 trucks	Cost at ₹0.10/MB
Full image pull (compressed)	120.652 MB	10,255.41 MB	₹1,025.54
Layer-cache update (model layer only)	0.00414 MB	0.3516 MB	₹0.035

Update method	Per truck	85 trucks	Cost at ₹0.10/MB

The final registry comparison reports a full-image transfer of 119.649 MB per truck versus a cache delta of 0.00573 MB per truck, a 99.9952% reduction across the assumed 85-truck fleet. The updated evidence is in evidence/latest_run_2026-11-03.txt. The 280 KB model size remains a separate assignment assumption for the OTA strategy calculation.

7. PSI drift monitoring (Task E1)

Definition. The monitor computes the Population Stability Index between a reference distribution and the last 100 inferences, every 60 s, and prints `[LOGIBRIDGE DRIFT ALERT] PSI=...` above 0.25. The reference is built from 300 clean Normal windows run through the deployed model and binned into four bins ([0, 0.25), [0.25, 0.5), [0.5, 0.75), [0.75, 1.0]); empty bins use a floor of 1e-4. The reference stores the model tag, and the monitor warns if the running model differs.

Confidence score. The score is the probability of the Normal class, not the maximum softmax probability. Table 3 shows the maximum softmax is about 1.0 for every class on this separable data, so PSI computed on it stays flat at 0.011 through the whole fault (Table 8), whereas P(Normal) falls from 0.995 to 0.000. [Confirm this interpretation of the brief's output confidence score with the instructor.]

Table 8: Offline simulation, M2 reference (anomaly from t = 600 s to 1,200 s)

t (s)	Windows	PSI on P(Normal)	PSI on max-softmax	Status
300	27	0.011	0.011	ok
600	57	0.011	0.011	ok
900	87	2.847	0.011	ALERT
1200	100	5.654	0.011	ALERT
1500	100	5.654	0.011	ALERT
1800	100	3.529	0.011	ALERT
2100	100	0.711	0.011	ALERT
2400	100	0.011	0.011	ok
2700	100	0.011	0.011	ok

The M2 reference had bin counts [0, 0, 1, 299] (one clean window scored 0.688); the baseline PSI of 0.011 comes from that single window.

Table 9: Live run through MQTT, M3 deployed (10x time acceleration, 30 s rehearsal interval)

Time	PSI	Inferences	Bin proportions	Status
06:59:52	0.000	46	0.00, 0.00, 0.00, 1.00	ok (before)

Time	PSI	Inferences	Bin proportions	Status
07:00:22	1.903	76	0.24, 0.00, 0.00, 0.76	ALERT (during)
07:00:52	4.382	100	0.48, 0.00, 0.00, 0.52	ALERT
07:01:22	5.648	100	0.59, 0.00, 0.00, 0.41	ALERT
07:01:52	4.717	100	0.51, 0.00, 0.00, 0.49	ALERT
07:02:22	1.655	100	0.21, 0.00, 0.00, 0.79	ALERT
07:02:52	0.000	100	0.00, 0.00, 0.00, 1.00	ok (after)

The first alert came at the first evaluation after injection, within 5 simulated minutes (about 27 s of real time at 10x). Recovery below 0.10 took about 1,170 simulated seconds after the anomaly ended, because the 100-inference window must flush. The large values (above 1) reflect the 1e-4 floor on empty reference bins through the ln(a/ε) term, so they mean far above 0.25 and not a precise effect size.

Drift type. The injected change is covariate shift: the input distribution moves (a sudden vibration step and an incremental temperature ramp) while the model's mapping is unchanged, and it is detected through the output-score distribution (prediction drift). PSI cannot tell a genuine fault from sensor degradation, so an alert should trigger review and not automatic retraining. We therefore treat drift alerts as a reason to inspect recorded operating conditions, not as an automatic retraining trigger.

Limitation observed. The monitor kept printing PSI 0.000 for hours after the simulator stopped, because it reads its last buffer and does not notice that no inferences are arriving.

8. Ansible deployment (Task E2)

Table 10: The seven tasks

#	Task	Module	Idempotency behaviour
1	Create /opt/logibridge	file	no change if it exists
2	Copy model to /opt/logibridge/model.tflite	copy	compares checksums
3	Copy reference_dist.json	copy	compares checksums
4	Stop the inference container if running	docker_container (absent)	runs only when task 2 or 3 changed; no-op if no container
5	Pull the image from the local registry	docker_image	pulls only if the tag is missing locally
6	Start the container with environment variables	docker_container	no change if image and settings match; recreated if they differ
7	Wait 15 s and verify the	shell	read-only check, never reports a

#	Task	Module	Idempotency behaviour
	container is running		change

The first attempt failed at task 4: `state: stopped` tries to create the container when it does not exist, so on a fresh host it failed with the message that a container cannot be created without an image. Using `state: absent` stops and removes the container if it exists and does nothing otherwise, which is what a stop-before-update step needs because task 6 recreates the container.

Table 11: Playbook runs

Run	Situation	Recap
3	Update to the M3 model and image 1.2 (earlier test run)	ok=7 changed=4 (model copied, reference copied, old container removed, new container started; the pull reported ok because the tag was already local). The final 4 November run used image 1.1 and reported ok=7 changed=3.
4	Immediate re-run, nothing changed	ok=6 changed=0 skipped=1 (the stop task is skipped)

Image tags are immutable per release, so a new release means a new tag.

9. OTA strategy selection (Task E3)

The analysis uses the brief's 280 KB model, ₹0.10 per MB, 85 trucks and an update every six weeks (about 8.7 cycles a year), with a canary group of 10, canary health telemetry of 4 reports a day of 1 KB for 3 days, and a shadow period of 7 days.

Table 12: Cost per update cycle

Scenario	MB per cycle	₹ per cycle	₹ per year	Trucks exposed to an unvalidated model
Full replacement	23.80	2.380	20.7	85
Full replacement, bad model, rollback by re-download	47.60	4.760	not applicable	85
Canary, success (10, then 75)	23.92	2.392	20.8	10
Canary, bad model caught, rollback to retained model	2.92	0.292	not applicable	10
Canary, bad model caught, rollback by re-download	5.72	0.572	not applicable	10
Shadow, on-device comparison	24.39	2.440	21.2	0

Scenario	MB per cycle	₹ per cycle	₹ per year	Trucks exposed to an unvalidated model
Shadow, upload every shadow output	537.88	53.788	467.4	0

Cost differs by a few paise, so risk decides. **A canary is recommended**: it limits a faulty model to 10 of 85 trucks, which matters because a missed Critical alert is the ₹28 lakh failure mode, and rollback to a model retained on the truck costs no bytes during 35 to 90 minute coverage gaps. Promotion requires Class 2 recall above 95% on a replay set and PSI below 0.25 over 48 to 72 hours. Full replacement exposes all 85 trucks at once, and coverage gaps delay the first sign of trouble. Shadow mode has no exposure, but it measures agreement with the current model and not correctness, may see no rare faults in its week, uses one to two weeks of a six-week cycle, and never exercises the real alert path; uploading all shadow outputs costs about 22 times as much. The real INT8 model is 3.4 KB, so an actual full replacement costs about ₹0.03.

10. Limitations and carry-forward items

- The simulated classes are trivially separable, so the 100% accuracy and recall say nothing about real faults.
- The confidence definition for PSI is an interpretation of the brief.
- The latest monitor includes a stale-inference check.
- The HTTP alert synchronizer is implemented in the latest demo, and the playbook assumes the network, broker and registry already exist.
- Carried to the final report: pruning (M3), benchmarking, the Pareto analysis and the deployment recommendation.

Final-run verification update — 3 November 2026

Our latest end-to-end test used real-time 1× sensor publishing (1,260 seconds), a combined fault during seconds 120–240, and PSI checks every 60 seconds on up to 100 inference results. Initial PSI was 0.000, the peak was 4.604, and the last evaluation returned to 0.000 with n=100. Two locally retained Critical alerts were delivered to the operations-centre receiver after it restarted. Ansible installed M3 and reported changed=0 on the second run. The OTA comparison calculated 99.9952% cached-layer saving for 85 trucks; this is not a measured cellular transfer. The original phase measurements remain as historical context; see reports/final_report.pdf and evidence/latest_run_2026-11-03.txt for final submission values.

Hardware cost and Roofline assumptions remain theoretical; all measured inference results are from synthetic data on a WSL laptop CPU. The current HTTP synchronizer is implemented, while real cellular modem testing and production TLS setup remain future work.

Final implementation and 4 November checks

This phase report records our earlier design and experiments. After the phase submission we completed and checked the live pipeline again on 4 November 2026. Our simulator ran for 1,260 seconds at normal speed; a combined anomaly lasted from second 120 to 240. With PSI checking every 60 real seconds over the last 100 predictions, we saw a clean 0.000 baseline, a peak of 4.604 and recovery to 0.000 with n=100. The operations centre received two alerts after we restarted it. Ansible completed the same seven-task deployment twice, reporting changed=0 the second time.

Our last CPU benchmark gave M1 mean/P95 0.00093/0.00109 ms, M2 0.00105/0.00112 ms and M3 0.00107/0.00108 ms. The model sizes remained 5.19, 3.38 and 2.94 KB, and synthetic Critical recall stayed at 100%. We preserved the earlier experiment figures but use the newer measurements in our final report. These tests used synthetic data and WSL, not real refrigerated trucks.